

```
import pandas as pd
import numpy as np

from io import StringIO

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncoder

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.feature_selection import SelectKBest, mutual_info_classif

from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression

from xgboost import XGBClassifier

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_auc_score
)

# =====
# LOAD DATASET
# =====

file_path = "BreastCancerDataset.xlsx"

raw = pd.read_excel(file_path, header=None)

csv_text = "\n".join(raw.iloc[:, 0].astype(str))

df = pd.read_csv(StringIO(csv_text))

print("Dataset Shape:", df.shape)

# =====
# CLEAN COLUMN NAMES
# =====

df.columns = df.columns.str.strip()

# =====
# FEATURES & TARGET
# =====

X = df.drop("CLASS", axis=1)
y = df["CLASS"]
```

```
# =====
# ENCODE TARGET
# =====

le = LabelEncoder()
y = le.fit_transform(y)

# =====
# CATEGORICAL & NUMERICAL COLUMNS
# =====

categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
numerical_cols = X.select_dtypes(exclude=['object']).columns.tolist()

# =====
# PREPROCESSING
# =====

numeric_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', numeric_transformer, numerical_cols),
    ('cat', categorical_transformer, categorical_cols)
])

# =====
# SPLIT DATA
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# =====
# BASE MODELS
# =====

svm_model = SVC(
    kernel='rbf',
    C=10,
    gamma='scale',
    probability=True,
```

```
        class_weight='balanced',
        random_state=42
    )

    rf_model = RandomForestClassifier(
        n_estimators=500,
        max_depth=15,
        min_samples_split=5,
        class_weight='balanced',
        random_state=42
    )

    xgb_model = XGBClassifier(
        n_estimators=500,
        max_depth=6,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        eval_metric='logloss',
        random_state=42
    )

    # =====
    # STACKING ENSEMBLE
    # =====

    stack_model = StackingClassifier(
        estimators=[
            ('svm', svm_model),
            ('rf', rf_model),
            ('xgb', xgb_model)
        ],
        final_estimator=LogisticRegression(max_iter=1000),
        cv=5
    )

    # =====
    # FINAL PIPELINE
    # =====

    model = Pipeline([
        ('preprocessing', preprocessor),
        ('classifier', stack_model)
    ])

    # =====
    # TRAIN
    # =====

    print("Training Advanced Stacking Ensemble...")

    model.fit(X_train, y_train)

    # =====
    # PREDICTION
```

```

# =====

y_pred = model.predict(X_test)

# =====
# ACCURACY
# =====

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", round(accuracy * 100, 2), "%")

# =====
# REPORT
# =====

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# =====
# CONFUSION MATRIX
# =====

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# =====
# ROC AUC
# =====

try:
    y_prob = model.predict_proba(X_test)

    if len(np.unique(y)) == 2:
        auc = roc_auc_score(y_test, y_prob[:, 1])
    else:
        auc = roc_auc_score(
            y_test,
            y_prob,
            multi_class='ovr'
        )

    print("\nROC-AUC:", round(auc, 4))

except:
    print("ROC-AUC could not be calculated.")

```

Dataset Shape: (2437, 23)
 Training Advanced Stacking Ensemble...

Accuracy: 93.65 %

Classification Report:

	precision	recall	f1-score	support
0	0.92	0.97	0.94	271

	1	0.96	0.90	0.93	217
accuracy				0.94	488
macro avg		0.94	0.93	0.94	488
weighted avg		0.94	0.94	0.94	488

Confusion Matrix:

```
[[262  9]
 [ 22 195]]
```

ROC-AUC: 0.9795

```
print("\nEnter Patient Details")

sex = int(input("Sex (0/1): "))
age = int(input("Age: "))
address = input("Address: ")
fbs = float(input("FBS: "))
urea = float(input("Urea: "))
creatinin = float(input("Creatinin: "))
alb = float(input("ALB: "))
t_ca = float(input("T_Ca: "))
gpt = float(input("GPT: "))
got = float(input("GOT: "))
alp = float(input("ALP: "))
ca153 = float(input("CA-15.3: "))
cea = float(input("CEA: "))
wbc = float(input("WBC: "))
rbc = float(input("RBC: "))
hgb = float(input("HGB: "))
plt = float(input("PLT: "))
esr = float(input("ESR: "))
ldh = float(input("LDH: "))
na = float(input("Na: "))
k = float(input("K: "))
cl = float(input("CL: "))
```

```
new_patient = pd.DataFrame({
    "Sex": [sex],
    "Age": [age],
    "Adress": [address],
    "FBS": [fbs],
    "Urea": [urea],
    "Creatinin": [creatinin],
    "ALB": [alb],
    "T_Ca": [t_ca],
    "GPT": [gpt],
    "GOT": [got],
    "ALP": [alp],
    "CA-15.3": [ca153],
    "CEA": [cea],
    "WBC": [wbc],
    "RBC": [rbc],
    "HGB": [hgb],
    "PLT": [plt].
```

```
"ESR": [esr],
"LDH": [ldh],
"Na": [na],
"K": [k],
"CL": [cl]
})

prediction = model.predict(new_patient)

print("\nPrediction:", le.inverse_transform(prediction)[0])
```

Enter Patient Details

Sex (0/1): 0

Age: 25

Address: Chennai

FBS: 90

Urea: 15

Creatinin: 0.9

ALB: 4

T_Ca: 9.2

GPT: 25

GOT: 20

ALP: 90

CA-15.3: 15

CEA: 2

WBC: 6.5

RBC: 4.5

HGB: 13.5

PLT: 250

ESR: 12

LDH: 200

Na: 140

K: 4

CL: 102

Prediction: 0

```
labels = {
    0: "Healthy",
    1: "Breast Cancer"
}
```

```
print("Prediction:", labels[prediction[0]])
```

Prediction: Healthy

```
probs = model.predict_proba(new_patient)

confidence = np.max(probs) * 100

print("Prediction:", labels[prediction[0]])
print("Confidence:", round(confidence, 2), "%")
```

Prediction: Healthy
Confidence: 96.0 %

```
prediction = model.predict(new_patient)

probabilities = model.predict_proba(new_patient)

cancer_probability = probabilities[0][1] * 100

if cancer_probability < 40:
    risk_level = "LOW RISK"
elif cancer_probability < 70:
    risk_level = "MEDIUM RISK"
else:
    risk_level = "HIGH RISK"

print("\n===== BREAST CANCER RISK ASSESSMENT =====")
print("Prediction:", prediction[0])
print("Cancer Probability:", round(cancer_probability, 2), "%")
print("Risk Level:", risk_level)
```

```
===== BREAST CANCER RISK ASSESSMENT =====
Prediction: 0
Cancer Probability: 4.0 %
Risk Level: LOW RISK
```